L6-v4

2026年10月18日 17:42

文件	变化类型	核心新增
sandbox.hpp	无变化	端口③契约早已定，首次实现
agent_loop.cpp	重大修改	六道闸门全部实现 + detectRepeat / injectWarning 定义
sandbox_win.cpp	新增文件	Windows 沙箱实现（Job Object + 临时文件接管）
main.cpp	重大修改	① Ctrl-C → stop_source；② Bash 换沙箱；③ 权限规则按 risk 生成；④ --sandbox-info；⑤ AF_CANCEL_AFTER_MS

L5 的循环只有"一道粗上限"

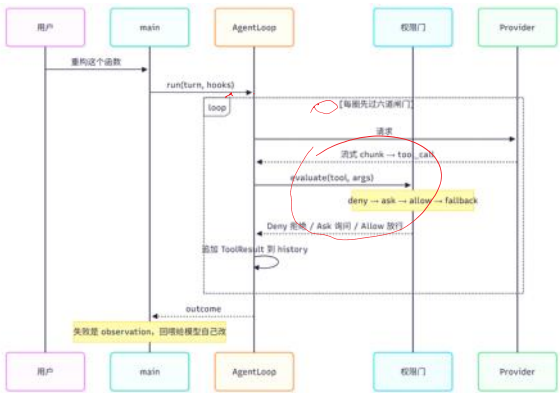
```
// L5 agent_loop.cpp
for (int step = 0; step < limits_.maxSteps; ++step) {
    // 组装 → 调模型 → 记账 → 结束判定 → 执行工具
    // 没有任何中途检查
}
return LoopOutcome::StepLimitReached;
```

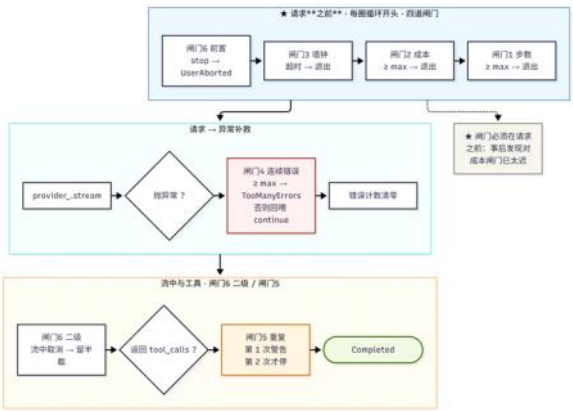
问题：

- 只能挡住"步数太多"，挡不住"很快但绕圈"、"很快但一直失败"、"很快但超预算"。
- 没有取消传播，用户按 Ctrl-C 只能杀进程。
- 没有重复检测，模型可能原样重发同一个调用。

新增六道闸门

闸门	字段	失败模式
1	maxSteps	模型在两个状态间来回横跳（A→B→A→B...）
2	maxTokensSpent	任务不可能完成，但模型永远"再试一次"
3	maxWallClock	一条卡住的命令把整个 turn 拖到永久
4	连续错误	地址/密钥配错（401/500），无限重试
5	重复调用	模型看不到自己的历史 → 原样重发同一个调用
6	用户中断	Ctrl-C 之后只能杀进程






```
AgentLoop::run(turn, hooks)
│
├── for (step = 0; step < maxSteps; ++step)    ← 闸门 1：步数
│   │
│   ├── [前置检查]                              ← 每次请求之前
│   │   ├── 闸门 6 前置：stop_requested()？
│   │   ├── 闸门 3：墙钟超时？
│   │   └── 闸门 2：token 预算耗尽？
│   │
│   ├── A. assembler_.assemble(turn)
│   │
│   ├── B. provider_.stream(req, onChunk)
│   │   │   │
│   │   │   └── onChunk 内：闸门 6 第一级（取消传到传输层）
│   │   │
│   │   └── catch: 闸门 4 连续错误
│   │
│   ├── [流结束后] 闸门 6 第二级（保留半截内容）
│   │
│   ├── C. 记账 + 存历史
│   │
```

```
for (int step = 0; step < limits_.maxSteps; ++step) {
    if (limits_.stop && limits_.stop->stop_requested()) {
        if (hooks.onNotice)
            hooks.onNotice("用户中断：停止（还没开始第 " +
                           std::to_string(step + 1) + " 步）");
        return LoopOutcome::UserAborted;
    }
    // ...
}
```

```
if (limits_.maxWallClock.count() > 0 &&
    std::chrono::duration_cast<Ms>(Clock::now() - tStart) >
        limits_.maxWallClock) {
    if (hooks.onNotice)
        hooks.onNotice("墙钟超时：已运行超过 " +
                       std::to_string(limits_.maxWallClock.count()) +
                       " ms，停止（闸门 3）");
    return LoopOutcome::WallClockExceeded;
```

```
│   │   └── catch: 闸门 4 连续错误
│   │
│   ├── [流结束后] 闸门 6 第二级（保留半截内容）
│   │
│   ├── C. 记账 + 存历史
│   │
│   ├── D. 无 toolCalls → Completed
│   │
│   └── E. for (tc : toolCalls)
│       │
│       ├── 闸门 5：detectRepeat(tc)?
│       │   ├── 第 1 次 → 警告 + 注入 tool 消息
│       │   └── 第 2 次 → StepLimitReached
│       │
│       └── registry_.dispatch(tc, ui)
│           │
│           └── Bash 工具 → sandbox_->exec(cmd, cwd, timeout)
│               │
│               ├── enabled=false → 拒绝执行
│               ├── policyCheck → 禁网 / denyRead
│               └── launch()
│                   ├── 临时文件接管 stdout/stderr
│                   ├── Job Object（生命周期 + 资源上限）
│                   ├── CreateProcessW(CREATE_SUSPENDED)
│                   ├── AssignProcessToJobObject
│                   ├── ResumeThread
│                   ├── WaitForSingleObject(timeout)
│                   │   └── 超时 → TerminateJobObject
│                   └── 关句柄 → 读临时文件 → 删临时文件
│
└── [for 结束] 闸门 1 触发 → StepLimitReached
```

两类护栏的分工：

层	组件	回答的问题	失效方式
循环层	六道闸门	"这个 turn 该不该继续"	逻辑漏判
执行层	沙箱	"这条命令到底能碰到什么"	内核约束（绕不过）

```
    if (hooks.onNotice)
        hooks.onNotice("墙钟超时：已运行超过 " +
                       std::to_string(limits_.maxWallClock.count()) +
                       " ms，停止（闸门 3）");
    return LoopOutcome::WallClockExceeded;
}
```

```
if (limits_.maxTokensSpent > 0 && spentTokens_ >= limits_.maxTokensSpent) {
    if (hooks.onNotice)
        hooks.onNotice("token 预算耗尽（" + std::to_string(spentTokens_) +
                       "/" + std::to_string(limits_.maxTokensSpent) +
                       "），停止（闸门 2）");
    return LoopOutcome::BudgetExceeded;
}
```

```
catch (const std::exception& e) {
    ++consecutiveErrors_;
    if (hooks.onError) hooks.onError(e.what());

    if (limits_.maxConsecutiveErrors > 0 &&
        consecutiveErrors_ >= limits_.maxConsecutiveErrors) {
        if (hooks.onNotice)
            hooks.onNotice("连续 " + std::to_string(consecutiveErrors_) +
                           " 次请求失败，停止（闸门 4）");
        return LoopOutcome::TooManyErrors;
    }

    if (hooks.onNotice)
        hooks.onNotice("模型请求失败（连续第 " +
                       std::to_string(consecutiveErrors_) +
                       " 次），已回喂给模型，继续下一步");

    // ★ 失败是 observation，不是"这个 turn 结束了"
    Message m;
    m.role    = Role::User;
    m.content = std::string("[系统] 模型请求失败：") + e.what() +
                "。请继续，或换一种方式。";
    turn.history.push_back(std::move(m));
    continue;
}
consecutiveErrors_ = 0;    // ★ 成功一次，连续计数归零
```

```
if (detectRepeat(tc)) {
    ++repeatStrikes;

    const ToolResult warn = ToolResult::error(tc.id, repeatWarningText(tc));
    if (hooks.onToolStart) hooks.onToolStart(tc);
    if (hooks.onToolDone)  hooks.onToolDone(warn);
    injectWarning(turn, tc);          // 回喂（保持 GB 合法性）
    ++done;

    if (hooks.onNotice)
        hooks.onNotice("检测到重复调用（第 " +
                       std::to_string(repeatStrikes) +
                       " 次），已插入警告而不是直接退出（闸门 5）");

    // ★ 已经给过机会了还不改 → 判定死循环。
    //   第 1 次：警告（软）。第 2 次：停（硬）。
    if (repeatStrikes > 1) {
        if (hooks.onNotice)
            hooks.onNotice("警告之后仍在重复调用，判定为死循环，停止（闸门 5）");
        return LoopOutcome::StepLimitReached;
    }
    continue;
}
```

```
std::deque<std::uint64_t> recentCallHashes_;
```

```
std::uint64_t hashCall(const ToolCall& c) {
    const std::string key = c.name + '\x1f' + c.args.dump();
    return static_cast<std::uint64_t>(std::hash<std::string>{}(key));
}
```

```
bool AgentLoop::detectRepeat(const ToolCall& call) {
    const std::uint64_t h = hashCall(call);

    // 数一数窗口里已经出现过几次（不含本次）
    std::size_t seen = 0;
    for (const auto prev : recentCallHashes_)
        if (prev == h) ++seen;

    recentCallHashes_.push_back(h);
    // 窗口上限 64：足够覆盖"读改读改"这种长交替，又不至于把很久以前的
    // 合法重复（例如隔了很多步又读一次同一个文件）算成死循环。
    while (recentCallHashes_.size() > 64) recentCallHashes_.pop_front();

    if (limits_.maxRepeatedCalls <= 0) return false;      // 0 = 关闭该闸门
    return static_cast<int>(seen) + 1 >= limits_.maxRepeatedCalls;
}
```

```
std::string repeatWarningText(const ToolCall& c) {
    return std::string("★ 重复调用被拦下（闸门 5）：你已经用**完全相同的参数**"
                       "调用过 `") + c.name + "` 若干次。\n"
                       "原样重试不会改变结果。请换一种做法：换参数、换工具，"
                       "或直接说明你卡在哪一步、需要什么信息。";
}
```

```
void AgentLoop::injectWarning(Turn& turn, const ToolCall& call) {
    Message m;
    m.role       = Role::Tool;
    m.toolCallId = call.id;
    m.content    = repeatWarningText(call);

    // ★ 警告必须以 **tool 角色**回喂，绝不能插一条 user 消息。
    //   原因在 GB（消息合法性）：assistant(tool_calls) 之后，
    //   在它的 tool 结果到达之前插入任何非 tool 消息，都会**关闭配对窗口**。
    //   于是那条姗姗来迟的 tool 结果变成孤儿 → 下一回请求直接 400。
    turn.history.push_back(std::move(m));
}
```

```
// main.cpp
std::stop_source g_stop;

#ifdef _WIN32
BOOL WINAPI onConsoleCtrl(DWORD type) {
    if (type == CTRL_C_EVENT || type == CTRL_BREAK_EVENT ||
        type == CTRL_CLOSE_EVENT) {
        g_stop.request_stop();
        // ★ 返回 TRUE = "我处理了，别立刻杀我"。
        //   返回 FALSE 的话进程被粗暴地终止，循环没机会收尾
        //   （已经写出去的文件、已经产生的 token 都白费）。
        return TRUE;
    }
    return FALSE;
}
#endif

// 注册
SetConsoleCtrlHandler(onConsoleCtrl, TRUE);

// 传给 LoopLimits
limits.stop = g_stop.get_token();
```

```
// 六道闸门
struct LoopLimits {
    int maxSteps           = 200;          // 闸门 1：步数上限
    int maxTokensSpent     = 2'000'000;    // 闸门 2：成本上限（必须有）
    Ms  maxWallClock{1'800'000};           // 闸门 3：墙钟
    int maxConsecutiveErrors = 5;          // 闸门 4：连续错误
    int maxRepeatedCalls     = 3;          // 闸门 5：死循环检测
    std::optional<std::stop_token> stop;   // 闸门 6：用户中断
};
```

```
resp = provider_.stream(req, [&](const ChatChunk& c) -> bool {
    if (hooks.onChunk) hooks.onChunk(c);

    // ---- 闸门 6 的第一级：把取消**一路传到传输层** --------
    //   返回 false 之后，调用链是：
    //     StreamCtx::emit → feed → postStream 回调
    //       → http_winhttp.cpp 的 onData 返回 false
    //       → doRequest 里 break，连接立刻关掉
    //   没有这条链，"用户按了 Ctrl-C 还要等模型把这句话说完"。
    if (limits_.stop && limits_.stop->stop_requested()) return false;
    return true;
});

// ---- 闸门 6 的第二级：取消发生在**流中途** ----------------------
//   此时已经收到部分内容。不丢弃它 —— 半截回答往往比"什么都没有"有用。
if (limits_.stop && limits_.stop->stop_requested()) {
    turn.finalText = resp.message.content;
    if (hooks.onNotice)
        hooks.onNotice("用户中断：已保留收到的 " +
                       std::to_string(resp.message.content.size()) + " 字符");
    return LoopOutcome::UserAborted;
}
```

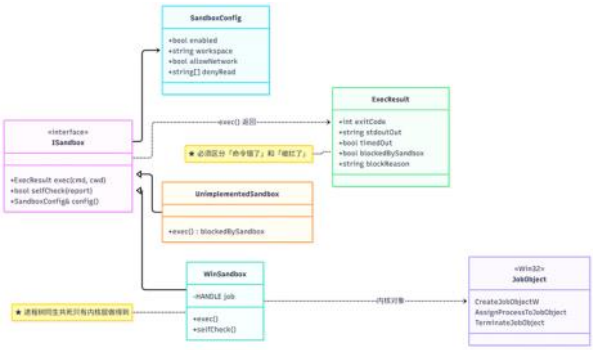
```
用户按 Ctrl-C
  → g_stop.request_stop()
    → stop_requested() == true
      → onChunk 返回 false
        → StreamCtx::emit 收到 false
          → feed 返回 false
            → postStream 的 onData 返回 false
              → WinHttpTransport::doRequest 里 cancelled = true; break
                → WinHttpReadData 循环结束
                  → 连接立刻关闭
```

沙箱实现

能力对照表：

能力	Linux (Landlock+seccomp)	Windows (本课实现
文件系统写隔离	✅内核强制，绕不过	❌未实现（需 AppC ）
网络隔离	✅内核强制	仅策略级（命令文本匹配，可绕过）

系统调用过滤	✅seccomp EPERM	❌未实现
进程树生命周期	✅cgroup	✅Job Object（内核强制）
资源上限	✅cgroup	✅Job Object（进程数 + 内存）
超时强杀	✅	✅TerminateJobObject



四层防护的执行顺序

```
exec(cmd, cwd, timeout)
  |
  ├─ ① enabled=false? → 拒绝执行（不静默降级）
  |
  ├─ ② policyCheck(cmd, cfg)
  |     ├─ allowNetwork=false → 检查命令文本是否含网络标记
  |     └─ denyRead → 检查命令文本是否含禁止路径模式
  |     命中 → blockedBySandbox = true
  |
  └─ ③ launch(cmd, cwd, timeout)
        ├─ 创建临时文件接管 stdout/stderr
        ├─ 创建 Job Object（KILL_ON_JOB_CLOSE + 进程数 + 内存）
        ├─ CreateProcessW(CREATE_SUSPENDED)
        ├─ AssignProcessToJobObject
        ├─ ResumeThread
        ├─ WaitForSingleObject(timeout)
        |    └─ 超时 → TerminateJobObject（连子进程一起杀）
        ├─ 关句柄 → 读临时文件 → 删临时文件
        └─ 返回 ExecResult
```

```cpp
PolicyVerdict policyCheck(const std::string& cmd, const SandboxConfig& cfg) {
    // ---- 禁网 ----
    if (!cfg.allowNetwork) {
        static const char* kNetMarkers[] = {
            "curl", "wget", "Invoke-WebRequest", "iwr ", "start-process",
            " ncat", "nc ", "ssh ", "scp ", "ftp ", "telnet",
            "http://", "https://",
        };
        for (const char* m : kNetMarkers) {
            if (cmd.find(m) != std::string::npos)
                return {true, std::string("网络已禁用（命令命中 `") + m + "`）"};
        }
    }

    // ---- denyRead ----
    for (const auto& p : cfg.denyRead) {
        if (!p.empty() && cmd.find(p) != std::string::npos)
            return {true, "命令命中了禁止访问的路径模式: " + p};
    }

    return {false, {}};
}
```

```cpp
HANDLE job = CreateJobObjectW(nullptr, nullptr);
JOBOBJECT_EXTENDED_LIMIT_INFORMATION li{};
li.BasicLimitInformation.LimitFlags =
    JOB_OBJECT_LIMIT_KILL_ON_JOB_CLOSE |      // 句柄一关，全树陪葬
    JOB_OBJECT_LIMIT_ACTIVE_PROCESS;
li.BasicLimitInformation.ActiveProcessLimit = kMaxProcs;   // 16
li.ProcessMemoryLimit                       = kMemLimitBytes;  // 1 GiB
SetInformationJobObject(job, JobObjectExtendedLimitInformation, &li, sizeof(li));
```

```cpp
const BOOL ok = CreateProcessW(
    nullptr, cmdline.data(), nullptr, nullptr,
    TRUE,                                          // 继承句柄
    CREATE_SUSPENDED | CREATE_NO_WINDOW | CREATE_NEW_PROCESS_GROUP,
    nullptr, wcwd.empty() ? nullptr : wcwd.c_str(), &si, &pi);

// ...

if (job) AssignProcessToJobObject(job, pi.hProcess);   // ★ 先挂 Job
ResumeThread(pi.hThread);                              // ★ 再放行
```

```cpp
const DWORD w = WaitForSingleObject(pi.hProcess, DWORD(ms));
if (w == WAIT_TIMEOUT) {
    r.timedOut = true;
    r.exitCode = 124;                  // 与 coreutils `timeout` 的约定一致
    if (job) TerminateJobObject(job, 124);          // ★ 连子进程一起杀
    else     TerminateProcess(pi.hProcess, 124);
    WaitForSingleObject(pi.hProcess, 5000);         // 给它 5s 死透
    r.blockReason = "命令超时（被沙箱强杀，含其子进程）";
}
```

```cpp
SECURITY_ATTRIBUTES sa{};
sa.nLength        = sizeof(sa);
sa.bInheritHandle = TRUE;              // 子进程要能继承这两个句柄

HANDLE hOut = CreateFileW(outPath.c_str(), GENERIC_WRITE, FILE_SHARE_READ, &sa,
                          CREATE_ALWAYS, FILE_ATTRIBUTE_NORMAL, nullptr);
HANDLE hErr = CreateFileW(errPath.c_str(), GENERIC_WRITE, FILE_SHARE_READ, &sa,
                          CREATE_ALWAYS, FILE_ATTRIBUTE_NORMAL, nullptr);
HANDLE hNul = CreateFileW(L"NUL", GENERIC_READ, FILE_SHARE_READ | FILE_SHARE_WRITE,
                          &sa, OPEN_EXISTING, FILE_ATTRIBUTE_NORMAL, nullptr);
```

三个句柄：

- `hOut`：stdout 重定向到临时文件。
- `hErr`：stderr 重定向到另一个临时文件。
- `hNul`：stdin 重定向到 `NUL`（空设备），防止命令等待用户输入。

`bInheritHandle = TRUE`：子进程要能继承这三个句柄，否则重定向不生效。

Bash 从 _popen 进沙箱

```cpp
// L5 registerBashTool
reg.registerBuiltin(def, [](const json& args) -> ToolResult {
    // ...
    FILE* pipe = _popen(cmd.c_str(), "r");
    // ...
});
```

```cpp
void registerBashTool(ToolRegistry& reg, std::shared_ptr<ISandbox> sandbox,
                      int timeoutMs) {
    // ... 定义 def ...
    reg.registerBuiltin(def, [sandbox, timeoutMs](const json& args) -> ToolResult {
        const std::string cmd = args.value("command", std::string());
        if (cmd.empty()) return ToolResult::error("", "command 不能为空");
        if (!sandbox) return ToolResult::error("", "沙箱未装配（拒绝执行，fail-closed）");

        std::error_code ec;
        const std::string cwd = pathUtf8(fs::current_path(ec));

        ExecResult er = sandbox->exec(cmd, cwd, Ms(timeoutMs));

        if (er.blockedBySandbox || er.timedOut)
            return ToolResult::error("", sandboxBlockedMessage(er));

        std::string out = er.stdoutOut;
        if (!er.stderrOut.empty()) out += "\n[stderr] " + er.stderrOut;
        if (er.exitCode != 0)
            return ToolResult::error("", out + "\n[exit code " + std::to_string(er.exitCode) + "]");
        if (out.empty()) out = "(命令成功，无输出)";
        return ToolResult::ok("", std::move(out));
    });
}
```

```cpp
// ★ 先关句柄再读文件：句柄不关，cmd 的缓冲可能还没 flush 到磁盘。
CloseHandle(hOut);
CloseHandle(hErr);
CloseHandle(hNul);

r.stdoutOut = readTextFile(outPath);
r.stderrOut = readTextFile(errPath);
DeleteFileW(outPath.c_str());
DeleteFileW(errPath.c_str());
```

```
├─ for (step = 0; step < maxSteps; ++step)   ← 闸门 1: 步数
│   │
│   ├─ [前置检查]                             ← 每次请求之前
│   │   ├─ 闸门 6 前置: stop_requested()?
│   │   ├─ 闸门 3: 墙钟超时?
│   │   └─ 闸门 2: token 预算耗尽?
│   │
│   ├─ A. assembler_.assemble(turn)
│   │
│   ├─ B. provider_.stream(req, onChunk)
│   │   │   │    │
│   │   │   │    └─ onChunk 内: 闸门 6 第一级 (取消传到传输层)
│   │   │   │
│   │   └─ catch: 闸门 4 连续错误
│   │
│   ├─ [流结束后] 闸门 6 第二级 (保留半截内容)
│   │
│   ├─ C. 记账 + 存历史
│   │
│   ├─ D. 无 toolCalls → Completed
│   │
│   └─ E. for (tc : toolCalls)
│       │
│       ├─ 闸门 5: detectRepeat(tc)?
│       │   ├─ 第 1 次 → 警告 + 注入 tool 消息
│       │   └─ 第 2 次 → StepLimitReached
│       │
│       └─ registry_.dispatch(tc, ui)
│           │
│           └─ Bash 工具 → sandbox_->exec(cmd, cwd, timeout)
│               │
│               ├─ enabled=false → 拒绝执行
│               ├─ policyCheck → 禁网 / denyRead
│               └─ launch()
│                   ├─ 临时文件接管 stdout/stderr
│                   ├─ Job Object (生命周期 + 资源上限)
│                   ├─ CreateProcessW(CREATE_SUSPENDED)
│                   ├─ AssignProcessToJobObject
│                   ├─ ResumeThread
│                   ├─ WaitForSingleObject(timeout)
│                   │   └─ 超时 → TerminateJobObject
│                   └─ 关句柄 → 读临时文件 → 删临时文件
│
└─ [for 结束] 闸门 1 触发 → StepLimitReached
```